Server-Sent Events (SSE)

You can stream data to the client using **Server-Sent Events** (SSE).

This is similar to Stream JSON Lines, but uses the `text/event-stream` format, which is supported natively by browsers with the `EventSource` API.

Info

Added in FastAPI `0.135.0`.

What are Server-Sent Events?

SSE is a standard for streaming data from the server to the client over HTTP.

Each event is a small text block with "fields" like `data`, `event`, `id`, and `retry`, separated by blank lines.

It looks like this:

```
data: {"name": "Portal Gun", "price": 999.99}

data: {"name": "Plumbus", "price": 32.99}
```

SSE is commonly used for AI chat streaming, live notifications, logs and observability, and other cases where the server pushes updates to the client.

Tip

If you want to stream binary data, for example video or audio, check the advanced guide: Stream Data.

Stream SSE with FastAPI

To stream SSE with FastAPI, use `yield` in your *path operation function* and set `response_class=EventSourceResponse`.

Import `EventSourceResponse` from `fastapi.sse`:

Python 3.10+

```python
from collections.abc import AsyncIterable, Iterable

from fastapi import FastAPI
from fastapi.sse import EventSourceResponse
from pydantic import BaseModel

app = FastAPI()


class Item(BaseModel):
    name: str
```

```python
    description: str | None


items = [
    Item(name="Plumbus", description="A multi-purpose household device."),
    Item(name="Portal Gun", description="A portal opening device."),
    Item(name="Meeseeks Box", description="A box that summons a Meeseeks."),
]


@app.get("/items/stream", response_class=EventSourceResponse)
async def sse_items() -> AsyncIterable[Item]:
    for item in items:
        yield item

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from collections.abc import AsyncIterable, Iterable

from fastapi import FastAPI
from fastapi.sse import EventSourceResponse
from pydantic import BaseModel

app = FastAPI()


class Item(BaseModel):
    name: str
    description: str | None


items = [
    Item(name="Plumbus", description="A multi-purpose household device."),
    Item(name="Portal Gun", description="A portal opening device."),
    Item(name="Meeseeks Box", description="A box that summons a Meeseeks."),
]


@app.get("/items/stream", response_class=EventSourceResponse)
async def sse_items() -> AsyncIterable[Item]:
    for item in items:
        yield item


@app.get("/items/stream-no-async", response_class=EventSourceResponse)
def sse_items_no_async() -> Iterable[Item]:
    for item in items:
        yield item


@app.get("/items/stream-no-annotation", response_class=EventSourceResponse)
async def sse_items_no_annotation():
    for item in items:
        yield item


@app.get("/items/stream-no-async-no-annotation", response_class=EventSourceResponse)
def sse_items_no_async_no_annotation():
    for item in items:
        yield item
```

Each yielded item is encoded as JSON and sent in the `data:` field of an SSE event.

If you declare the return type as `AsyncIterable[Item]`, FastAPI will use it to **validate**, **document**, and **serialize** the data using Pydantic.

Python 3.10+

```python
from collections.abc import AsyncIterable, Iterable

from fastapi import FastAPI
from fastapi.sse import EventSourceResponse
from pydantic import BaseModel

app = FastAPI()


class Item(BaseModel):
    name: str
    description: str | None


items = [
    Item(name="Plumbus", description="A multi-purpose household device."),
    Item(name="Portal Gun", description="A portal opening device."),
    Item(name="Meeseeks Box", description="A box that summons a Meeseeks."),
]


@app.get("/items/stream", response_class=EventSourceResponse)
async def sse_items() -> AsyncIterable[Item]:
    for item in items:
        yield item

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from collections.abc import AsyncIterable, Iterable

from fastapi import FastAPI
from fastapi.sse import EventSourceResponse
from pydantic import BaseModel

app = FastAPI()


class Item(BaseModel):
    name: str
    description: str | None


items = [
    Item(name="Plumbus", description="A multi-purpose household device."),
    Item(name="Portal Gun", description="A portal opening device."),
    Item(name="Meeseeks Box", description="A box that summons a Meeseeks."),
]


@app.get("/items/stream", response_class=EventSourceResponse)
async def sse_items() -> AsyncIterable[Item]:
    for item in items:
        yield item


@app.get("/items/stream-no-async", response_class=EventSourceResponse)
def sse_items_no_async() -> Iterable[Item]:
    for item in items:
        yield item


@app.get("/items/stream-no-annotation", response_class=EventSourceResponse)
async def sse_items_no_annotation():
    for item in items:
        yield item


@app.get("/items/stream-no-async-no-annotation", response_class=EventSourceResponse)
def sse_items_no_async_no_annotation():
    for item in items:
        yield item
```

Tip

As Pydantic will serialize it in the **Rust** side, you will get much higher **performance** than if you don't declare a return type.

Non-async *path operation functions*

You can also use regular `def` functions (without `async`), and use `yield` the same way.

FastAPI will make sure it's run correctly so that it doesn't block the event loop.

As in this case the function is not async, the right return type would be `Iterable[Item]`:

Python 3.10+

```python
# Code above omitted 👆

@app.get("/items/stream-no-async", response_class=EventSourceResponse)
def sse_items_no_async() -> Iterable[Item]:
```

```
    for item in items:
        yield item

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from collections.abc import AsyncIterable, Iterable

from fastapi import FastAPI
from fastapi.sse import EventSourceResponse
from pydantic import BaseModel

app = FastAPI()


class Item(BaseModel):
    name: str
    description: str | None


items = [
    Item(name="Plumbus", description="A multi-purpose household device."),
    Item(name="Portal Gun", description="A portal opening device."),
    Item(name="Meeseeks Box", description="A box that summons a Meeseeks."),
]


@app.get("/items/stream", response_class=EventSourceResponse)
async def sse_items() -> AsyncIterable[Item]:
    for item in items:
        yield item


@app.get("/items/stream-no-async", response_class=EventSourceResponse)
def sse_items_no_async() -> Iterable[Item]:
    for item in items:
        yield item


@app.get("/items/stream-no-annotation", response_class=EventSourceResponse)
async def sse_items_no_annotation():
    for item in items:
        yield item


@app.get("/items/stream-no-async-no-annotation", response_class=EventSourceResponse)
def sse_items_no_async_no_annotation():
    for item in items:
        yield item
```

No Return Type

You can also omit the return type. FastAPI will use the `jsonable_encoder` to convert the data and send it.

Python 3.10+

```python
# Code above omitted 👆

@app.get("/items/stream-no-annotation", response_class=EventSourceResponse)
async def sse_items_no_annotation():
    for item in items:
        yield item

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from collections.abc import AsyncIterable, Iterable

from fastapi import FastAPI
from fastapi.sse import EventSourceResponse
from pydantic import BaseModel

app = FastAPI()


class Item(BaseModel):
    name: str
    description: str | None


items = [
    Item(name="Plumbus", description="A multi-purpose household device."),
    Item(name="Portal Gun", description="A portal opening device."),
    Item(name="Meeseeks Box", description="A box that summons a Meeseeks."),
]


@app.get("/items/stream", response_class=EventSourceResponse)
async def sse_items() -> AsyncIterable[Item]:
    for item in items:
        yield item


@app.get("/items/stream-no-async", response_class=EventSourceResponse)
def sse_items_no_async() -> Iterable[Item]:
    for item in items:
        yield item


@app.get("/items/stream-no-annotation", response_class=EventSourceResponse)
async def sse_items_no_annotation():
    for item in items:
        yield item


@app.get("/items/stream-no-async-no-annotation", response_class=EventSourceResponse)
def sse_items_no_async_no_annotation():
    for item in items:
        yield item
```

`ServerSentEvent`

If you need to set SSE fields like `event`, `id`, `retry`, or `comment`, you can yield `ServerSentEvent` objects instead of plain data.

Import `ServerSentEvent` from `fastapi.sse`:

Python 3.10+

```python
from collections.abc import AsyncIterable

from fastapi import FastAPI
from fastapi.sse import EventSourceResponse, ServerSentEvent
from pydantic import BaseModel

app = FastAPI()


class Item(BaseModel):
```

```
    name: str
    price: float


items = [
    Item(name="Plumbus", price=32.99),
    Item(name="Portal Gun", price=999.99),
    Item(name="Meeseeks Box", price=49.99),
]


@app.get("/items/stream", response_class=EventSourceResponse)
async def stream_items() -> AsyncIterable[ServerSentEvent]:
    yield ServerSentEvent(comment="stream of item updates")
    for i, item in enumerate(items):
        yield ServerSentEvent(data=item, event="item_update", id=str(i + 1), retry=5000)
```

The `data` field is always encoded as JSON. You can pass any value that can be serialized as JSON, including Pydantic models.

Raw Data

If you need to send data **without** JSON encoding, use `raw_data` instead of `data`.

This is useful for sending pre-formatted text, log lines, or special *"sentinel"* values like `[DONE]`.

Python 3.10+

```
from collections.abc import AsyncIterable

from fastapi import FastAPI
from fastapi.sse import EventSourceResponse, ServerSentEvent

app = FastAPI()


@app.get("/logs/stream", response_class=EventSourceResponse)
async def stream_logs() -> AsyncIterable[ServerSentEvent]:
    logs = [
        "2025-01-01 INFO  Application started",
        "2025-01-01 DEBUG Connected to database",
        "2025-01-01 WARN  High memory usage detected",
    ]
    for log_line in logs:
        yield ServerSentEvent(raw_data=log_line)
```

Note

`data` and `raw_data` are mutually exclusive. You can only set one of them on each `ServerSentEvent`.

Resuming with `Last-Event-ID`

When a browser reconnects after a connection drop, it sends the last received `id` in the `Last-Event-ID` header.

You can read it as a header parameter and use it to resume the stream from where the client left off:

Python 3.10+

```
from collections.abc import AsyncIterable
from typing import Annotated
```

```python
from fastapi import FastAPI, Header
from fastapi.sse import EventSourceResponse, ServerSentEvent
from pydantic import BaseModel

app = FastAPI()


class Item(BaseModel):
    name: str
    price: float


items = [
    Item(name="Plumbus", price=32.99),
    Item(name="Portal Gun", price=999.99),
    Item(name="Meeseeks Box", price=49.99),
]


@app.get("/items/stream", response_class=EventSourceResponse)
async def stream_items(
    last_event_id: Annotated[int | None, Header()] = None,
) -> AsyncIterable[ServerSentEvent]:
    start = last_event_id + 1 if last_event_id is not None else 0
    for i, item in enumerate(items):
        if i < start:
            continue
        yield ServerSentEvent(data=item, id=str(i))
```

SSE with POST

SSE works with **any HTTP method**, not just `GET`.

This is useful for protocols like MCP that stream SSE over `POST`:

Python 3.10+

```python
from collections.abc import AsyncIterable

from fastapi import FastAPI
from fastapi.sse import EventSourceResponse, ServerSentEvent
from pydantic import BaseModel

app = FastAPI()


class Prompt(BaseModel):
    text: str


@app.post("/chat/stream", response_class=EventSourceResponse)
async def stream_chat(prompt: Prompt) -> AsyncIterable[ServerSentEvent]:
    words = prompt.text.split()
    for word in words:
        yield ServerSentEvent(data=word, event="token")
    yield ServerSentEvent(raw_data="[DONE]", event="done")
```

Technical Details

FastAPI implements some SSE best practices out of the box.

- Send a **"keep alive" `ping` comment** every `15` seconds when there hasn't been any message, to prevent some proxies from closing the connection, as suggested in the HTML specification: Server-Sent Events.
- Set the `Cache-Control: no-cache` header to **prevent caching** of the stream.

- Set a special header `X-Accel-Buffering: no` to **prevent buffering** in some proxies like Nginx.

You don't have to do anything about it, it works out of the box. 🤓